

Tema 3 – Ingeniería de Requisitos

Capa de síntesis. No sustituye a los apuntes (referencia) ni a los tests/H5P (evaluación): sirve para repasar antes de usarlos.

Nota sobre la numeración de esta hoja: los objetivos de aprendizaje se numeran OA1–OA10 y los bloques de esenciales, Bloque 1–Bloque 10, para no confundirlos con la numeración jerárquica de los apuntes (3.1–3.5, con subapartados). Cada bloque indica entre paréntesis el apartado de los apuntes al que corresponde.

Objetivos de aprendizaje

OA1. Justificar la importancia de una captura rigurosa de requisitos para el éxito de un proyecto software, e identificar las consecuencias típicas de una captura deficiente.

(apuntes: 3.1.1–3.1.2)

OA2. Explicar la ingeniería de requisitos como proceso (elicitación, análisis, especificación, validación, gestión) y describir el papel del analista de requisitos como mediador entre stakeholders. *(apuntes: 3.1.5)*

OA3. Distinguir los cinco tipos de requisitos de la taxonomía de Wiegers y Beatty (BO, UR, FR, NFR, BR) y explicar cómo se derivan unos de otros. *(apuntes: 3.1.3)*

OA4. Enumerar los tipos de requisitos no funcionales (atributos de calidad, restricciones, interfaces externas) y diferenciar una restricción real de una idea de solución.

(apuntes: 3.1.3.4)

OA5. Aplicar las características de calidad de un requisito según la norma ISO/IEC/IEEE 29148:2018 para evaluar y corregir su redacción. *(apuntes: 3.1.3.4 características + 3.4.3 checklist)*

OA6. Identificar los tipos de stakeholders de un proyecto, distinguir técnicas de detección de conflictos frente a técnicas de negociación, y aplicar técnicas de priorización colaborativa. *(apuntes: 3.1.4 + 3.4.1 + 3.4.2)*

OA7. Seleccionar la técnica de elicitación de requisitos más adecuada (entrevista, taller, observación, cuestionario, análisis documental...) según el contexto del proyecto.

(apuntes: 3.2)

OA8. Redactar una historia de usuario completa (narrativa, criterios de aceptación, Definición de Hecho) siguiendo las convenciones habituales en herramientas ágiles como Jira. *(apuntes: 3.3.6)*

OA9. Modelar un conjunto de casos de uso identificando actores, relaciones (inclusión, extensión, generalización) y escenarios principal y alternativos. (*apuntes: 3.3.2–3.3.5*)

OA10. Explicar el papel de la trazabilidad y la gestión de cambios en el ciclo de vida de los requisitos, y distinguir las herramientas habituales de gestión. (*apuntes: 3.4.4 + 3.5*)

Esenciales por bloque

Bloque 1 · Importancia y concepto de requisito (apuntes: 3.1.1–3.1.2)

- Un requisito es una guía detallada de lo que debe implementarse en un sistema: define comportamiento, funciones, atributos y restricciones (técnicas, legales, de tiempo).
- Los requisitos guían todo el ciclo de vida: planificación, diseño, implementación, validación, mantenimiento y evolución.
- **Efecto cascada:** un requisito mal definido genera defectos de diseño, que producen errores de implementación y fallos en pruebas; corregir un error en producción puede costar varios órdenes de magnitud más que corregirlo en la fase de requisitos.
- Casos reales que ilustran el impacto de una captura deficiente:
 - **Aeropuerto de Denver (1994-95):** sistema automatizado de equipajes, sobrecoste superior a 560 millones de dólares por falta de participación de usuarios clave, requisitos cambiantes sin control y captura deficiente del entorno físico y logístico.
 - **Sistema de venta de entradas de los JJ.OO. de Londres 2012:** requisitos de rendimiento y pruebas de carga insuficientes.
 - **RBS (2012):** sistemas heredados sin requisitos explícitos de disponibilidad.
 - *Concord* (Sony, 2024): retirado 11 días tras el lanzamiento por una captura deficiente de los requisitos de negocio y de mercado.
 - **IRS (Internal Revenue Service):** proyecto de modernización con sobrecostes acumulados que llegaron a superar los 4.000 millones de dólares, la cifra más alta entre los casos que registran los apuntes.
 - **Voto electrónico alemán (2009):** sistema declarado inconstitucional por falta de garantías de verificabilidad; ejemplo de fallo por requisitos no funcionales (transparencia, auditabilidad) no capturados adecuadamente.
- Consecuencias típicas de una captura deficiente: sobrecostes por cambios tardíos, retrasos en las entregas, insatisfacción del cliente y deterioro de la moral del equipo.

Bloque 2 · La ingeniería de requisitos como proceso · rol del analista (apuntes: 3.1.5)

- La **ingeniería de requisitos** es el proceso sistemático que abarca todas las actividades relacionadas con los requisitos: **elicitación** (descubrir), **análisis** (depurar y resolver inconsistencias), **especificación** (documentar), **validación** (asegurar que reflejan las necesidades reales) y **gestión** (mantener el conjunto vivo a lo largo del ciclo de vida). No se limita a "recoger" requisitos: los construye activamente en colaboración con los stakeholders.

- El **analista de requisitos** es el rol profesional encargado de conducir el proceso. Actúa como mediador entre los stakeholders (clientes, usuarios, negocio, legal, desarrollo), y como puente entre el dominio del problema y el dominio de la solución. Sus responsabilidades típicas: identificar y clasificar stakeholders, elegir y aplicar técnicas de elicitación, redactar y revisar la especificación, facilitar la resolución de conflictos, y mantener la trazabilidad de los requisitos.
- El resto de bloques de este tema desarrollan cada una de las actividades del proceso; conviene tener presente que forman un ciclo iterativo, no una secuencia estrictamente lineal.

Bloque 3 · Taxonomía de requisitos (BO, UR, FR, NFR, BR) (apuntes: 3.1.3)

- La asignatura sigue la clasificación de **Wieggers y Beatty** (*Software Requirements*), con prefijos alineados con las recomendaciones de IEEE y con herramientas como Jira.
- **BO — Requisitos u objetivos de negocio:** directrices de alto nivel definidas por patrocinadores; responden al *por qué* del proyecto; se documentan en el Documento de Visión y Alcance. Ejemplo: BO-01, reducir un 25 % los costes operativos en los mostradores del aeropuerto.
- **UR — Requisitos de usuario:** necesidades de los usuarios finales, a alto nivel; responden a *qué debe poder hacer el usuario*; se documentan como casos de uso o historias de usuario.
- **FR — Requisitos funcionales:** comportamiento específico del sistema, derivado directamente de los UR (ejemplo: el UR "el cliente debe poder pagar en línea" se traduce en el FR "el sistema debe procesar las transacciones a través de pasarelas seguras").
- **NFR — Requisitos no funcionales:** características o restricciones sobre *cómo* debe comportarse el sistema (a diferencia de los FR, que dicen *qué* debe hacer); se subdividen en atributos de calidad, restricciones e interfaces externas (ver Bloque 4).
- **BR — Reglas de negocio:** directrices que gobiernan el comportamiento organizativo, con origen en el negocio y no en el sistema (ejemplo: descuento del 10 % a clientes con más de 5 años de antigüedad).
- Cadena de derivación habitual: BO → UR → FR, con los NFR y las BR condicionando el conjunto.
- Distinción adicional: **requisitos de producto** (comportamiento observable del sistema final) frente a **requisitos de proyecto** (calendarios, presupuestos, recursos, estándares organizativos).

Bloque 4 · Requisitos no funcionales: calidad, restricciones e interfaces (apuntes: 3.1.3.4)

- **Atributos de calidad** (los apuntes recogen 15 atributos como lista plana; a continuación se presentan agrupados por afinidad — *elaboración derivada; ver disclaimer*):

- Rendimiento y capacidad: rendimiento (*performance*), eficiencia en el uso de recursos, escalabilidad ante el crecimiento de carga.
- Fiabilidad y continuidad: fiabilidad, disponibilidad, robustez ante entradas inválidas, seguridad funcional (*safety*) en sistemas críticos.
- Seguridad e integridad de los datos: seguridad (*security*), integridad.
- Calidad de uso: usabilidad, portabilidad entre entornos, instalabilidad.
- Mantenimiento y verificación: modificabilidad, verificabilidad o testabilidad, reusabilidad.
- Se recomienda redactarlos con criterios **SMART** (específicos, medibles, alcanzables, relevantes, acotados en el tiempo).
- **Restricciones:** limitaciones externas de origen tecnológico, de hardware, normativo, de compatibilidad, de interfaces existentes o presupuestario/de gestión.
- Distinguir una **restricción real** de una **idea de solución**: si un usuario pide "una lista desplegable", eso no es un requisito sino una propuesta de implementación; preguntar reiteradamente "¿por qué?" ayuda a descubrir la necesidad real detrás. No separarlas produce funcionalidad "chapada en oro" (*gold plating*), restricciones innecesarias y automatización de procesos ineficaces.
- **Requisitos de interfaz externa:** de usuario (pantallas, accesibilidad), de software (APIs, bases de datos, servicios externos), de hardware (sensores, impresoras) y de comunicación (protocolos de red).

Bloque 5 · Calidad de los requisitos: características

normalizadas y revisión (apuntes: 3.1.3.4 características + 3.4.3 checklist)

- La norma **ISO/IEC/IEEE 29148:2018** define nueve características que debe cumplir todo requisito bien redactado: necesario, apropiado (nivel de abstracción adecuado: el *qué*, no el *cómo*), no ambiguo, completo, singular (una única capacidad por requisito), factible, verificable, correcto y conforme a la plantilla de la organización.
- Las que con más frecuencia falla el alumnado novel son *no ambiguo, verificable y singular*: conviene revisarlas primero.
- **Checklist de revisión** (los apuntes recogen 12 comprobaciones como lista plana; a continuación se presentan agrupadas por eje — *elaboración derivada; ver disclaimer*):
 - Redacción: claridad, precisión (evitar términos vagos como "rápido" o "fácil" sin cuantificar), formato correcto ("el sistema debe...").
 - Contenido del requisito: completitud, unicidad (no duplicado), independencia (se entiende por sí solo).
 - Relación con el resto del sistema: consistencia, trazabilidad, necesidad real.
 - Viabilidad y gestión: verificabilidad, factibilidad, prioridad definida.

- Niveles de formalidad en la revisión: inspecciones formales con roles definidos (proyectos críticos o regulados), checklists de calidad inspiradas en la norma anterior, y revisiones informales entre pares (proyectos pequeños o fases tempranas).
- **Análisis** de requisitos: depurar inconsistencias, ambigüedades y redundancias.
Validación: asegurar que reflejan fielmente las necesidades reales de negocio y usuario.

Bloque 6 · Stakeholders: identificación, conflictos y priorización (apuntes: 3.1.4 + 3.4.1 + 3.4.2)

- Tipos habituales de stakeholders: clientes, usuarios finales, expertos del dominio, desarrolladores y diseñadores, personal de soporte y operaciones, responsables legales o de cumplimiento, y stakeholders externos (otros sistemas, organismos, proveedores).
- El analista de requisitos actúa como facilitador y mediador ante conflictos típicos (usuario frente a cliente en la complejidad de la interfaz, negocio frente a legal, entrega temprana frente a calidad).
- **Detección de conflictos** (previa a la negociación): comparación sistemática de requisitos entre sí, análisis de dependencias, entrevistas de validación cruzada con distintos stakeholders. Detectar el conflicto lo antes posible reduce mucho su coste de resolución.
- Técnicas de **priorización colaborativa** entre stakeholders:
 - **MoSCoW** (Must / Should / Could / Won't): clasifica cada requisito en una de las cuatro categorías; útil para acotar alcance en proyectos ágiles.
 - **Reparto de 100 puntos** entre requisitos: cada stakeholder distribuye 100 puntos según su percepción de valor; permite comparar prioridades entre partes con criterios distintos.
 - **Análisis valor/coste:** cruza el valor esperado con el esfuerzo estimado; los requisitos de alto valor y bajo coste se priorizan.
 - **Modelo de Kano** (centrado en la satisfacción del usuario), con tres subcategorías principales de requisitos: **must-be** (básicos: su ausencia produce insatisfacción, su presencia no aporta satisfacción explícita); **one-dimensional** (rendimiento: satisfacción proporcional al grado de cumplimiento); **delighters** (encantadores: no esperados, generan satisfacción desproporcionada si están, no producen insatisfacción si faltan).
 - **Pareto (80/20):** identifica el 20 % de requisitos que aporta el 80 % del valor, para reducir alcance sin comprometer el resultado.
- Buenas prácticas de **negociación:** identificar conflictos cuanto antes, documentar los acuerdos, mantener comunicación continua y designar responsables de la decisión final.

Bloque 7 · Técnicas de elicitación de requisitos (apuntes: 3.2)

- Elicitar no es simplemente "recoger" información disponible: es descubrir y construir conocimiento de forma activa, incluyendo requisitos implícitos que el usuario no verbaliza (por ejemplo, la seguridad en un sistema de comercio electrónico).
- Nueve técnicas, cada una con su contexto de uso preferente:
 - **Entrevistas:** individuales o grupales; estructuradas para requisitos generales iniciales, semiestructuradas para explorar aspectos específicos, no estructuradas para exploración abierta.
 - **Reuniones estructuradas:** objetivo claro, agenda, moderador y registro de conclusiones; útiles para revisar el documento de requisitos y detectar inconsistencias.
 - **Lluvia de ideas (brainstorming):** no juzgar durante la generación, priorizar cantidad, construir sobre ideas ajenas y evaluar después.
 - **Talleres de requisitos (workshops):** sesión facilitada con objetivo y agenda concretos, apoyada en prototipos o borradores previos.
 - **Observación directa:** pasiva o interactiva; especialmente útil cuando el sistema actual ya está en uso o los usuarios no son expertos en tecnología.
 - **Cuestionarios y encuestas:** cuando hay muchos usuarios distribuidos y las entrevistas individuales no son viables; conviene pilotarlos antes de distribuirlos.
 - **Análisis de interfaces de sistema:** identifica cómo debe interactuar el sistema con otros sistemas externos (interoperabilidad).
 - **Análisis de interfaces de usuario:** examina sistemas ya existentes, propios o de la competencia, para trasladar o mejorar elementos.
 - **Análisis de documentación:** especificaciones previas, manuales, normativas; riesgo principal, la obsolescencia de lo revisado.
- En la práctica profesional inmediata de un ingeniero de software junior, las entrevistas y los cuestionarios son, con diferencia, las técnicas de uso más frecuente.

Bloque 8 · Historias de usuario: formato, componentes y buenas prácticas (apuntes: 3.3.6)

- Formato de la narrativa: **"Como [rol de usuario], necesito [función], para [beneficio]"**. Cada historia es "el inicio de una conversación", no un requisito cerrado.
- Componentes: título breve, narrativa, criterios de aceptación, Definición de Hecho y, opcionalmente, prioridad y estimación de esfuerzo (técnica habitual: *planning poker*).
- **Criterios de aceptación** frente a **Definición de Hecho (DoD, Definition of Done)**: los criterios de aceptación son específicos de cada historia; la DoD es un acuerdo genérico y compartido por todo el equipo sobre qué significa "terminado" (por ejemplo, cobertura de pruebas mínima, revisión de código, ausencia de errores bloqueantes). Cumplir los criterios de aceptación no basta si no se cumple también la DoD.

- Formato opcional **Given/When/Then** (Gherkin): *Given* fija el contexto, *When* la acción que dispara el comportamiento, *Then* el resultado esperado; se pueden encadenar cláusulas con *And/But*. Recomendable con BDD y herramientas como Cucumber o SpecFlow; no conviene para requisitos técnicos sin un camino de usuario claro, reglas de negocio estáticas, restricciones de diseño visual o requisitos cuantitativos (mejor una tabla en esos casos). Su uso es opcional y combinable con lenguaje natural.
- Características de una buena historia: brevedad y simplicidad, enfoque en el usuario, independencia (implementable sin depender de otras historias) y valor claro para el cliente.
- Limitaciones: dependen de una buena colaboración con el cliente, la priorización se complica con múltiples stakeholders, y su lenguaje sencillo puede generar interpretaciones ambiguas si faltan detalles técnicos.
- Las historias de usuario son el formato habitual de las herramientas ágiles de gestión de backlog (por ejemplo, Jira).

Bloque 9 · Casos de uso: actores, relaciones y especificación (apuntes: 3.3.2–3.3.5)

- Un caso de uso describe cómo un actor interactúa con el sistema para lograr un objetivo; es, ante todo, un documento, no solo un diagrama. Fueron introducidos por **Ivar Jacobson**.
- **Actor**: rol abstracto (persona, sistema o dispositivo), distinto del usuario concreto que lo encarna: la metáfora del "sombrero" ilustra que una misma persona puede representar varios actores según la tarea. Nomenclatura UML: mayúscula inicial y singular.
- Tipos de actores: principal (inicia el caso de uso), secundario o de apoyo (da un servicio necesario), pasivo (recibe información sin iniciar nada), abstracto (generaliza otros actores) e iniciador (desencadena el trabajo de otro actor sin usar el sistema; no contemplado en el estándar UML).
- **Relaciones entre casos de uso**: inclusión (<<include>>, obligatoria: un caso de uso siempre necesita la funcionalidad del incluido), extensión (<<extend>>, opcional: solo se activa bajo ciertas condiciones) y generalización (análoga a la herencia, aplicable también a actores).
- Técnicas de identificación de casos de uso: a partir de actores, mediante escenarios generalizados, análisis de procesos de negocio, eventos externos, análisis CRUD, objetivos de usuario, a partir de los requisitos de usuario y funcionales (la técnica que se aplica en las clases de prácticas de la asignatura) y, como excepción, agrupando operaciones repetitivas de alta/baja/modificación/consulta en un único caso de uso.
- Cada caso de uso se compone de un **escenario principal** (flujo básico) y **escenarios alternativos** (variaciones, excepciones, errores), junto con sus precondiciones y postcondiciones.

- Buenas prácticas de modelado: mantener un nivel de abstracción suficiente (el *qué*, no el *cómo*), alcance bien definido, granularidad adecuada (procesos de negocio elementales o EBP), y priorizar siempre la descripción textual sobre el diagrama, que por sí solo no captura todos los detalles necesarios para el desarrollo y las pruebas.

Bloque 10 · Trazabilidad, gestión de cambios y herramientas (apuntes: 3.4.4 + 3.5)

- La **trazabilidad** permite seguir el rastro de un requisito durante todo su ciclo de vida: de dónde surge, dónde está implementado, qué pruebas lo validan y qué ocurre si cambia. Relaciona típicamente requisitos con casos de uso o historias de usuario, con tests de aceptación y con elementos de diseño o código.
- Beneficios de la trazabilidad: evita requisitos olvidados, facilita auditorías y certificaciones, permite reaccionar a cambios normativos o tecnológicos, y da confianza a los stakeholders.
- **Control de cambios:** proceso para evitar el *scope creep* (crecimiento incontrolado del alcance), alineado con la gestión de configuración del Tema 2. Secuencia habitual: solicitud de cambio, registro y categorización, análisis de impacto, decisión por un Change Control Board (CCB), implementación y actualización de artefactos, y seguimiento y auditoría.
- Herramientas de gestión de requisitos: **Jira** (orientada a ágil: historias de usuario, backlog, tableros Kanban/Scrum), **Redmine** (open source, adecuada para presupuestos ajustados) e **IBM DOORS** (estándar en sectores críticos como aeronáutica o automoción, con trazabilidad integral y asociada a normativas como DO-178C o ISO 26262).

Tabla de decisión: ¿historia de usuario o caso de uso completo?

Situación del proyecto	Elección recomendada	Por qué
Proyecto ágil, equipo pequeño, entregas incrementales frecuentes	Historia de usuario	Ligereza y foco en el valor entregado; permite iterar sin sobrecarga documental.
Sistema con flujos alternativos, excepciones y reglas de negocio complejas	Caso de uso completo	Necesita capturar el flujo principal y los alternativos, con precondiciones y postcondiciones explícitas.
Entorno regulado (sanitario, aeronáutico, financiero, administración pública)	Caso de uso completo, con trazabilidad formal	Exigencia normativa de verificabilidad y auditoría (por ejemplo, DO-178C, ISO 26262, ISO/IEC/IEEE 29148).
Stakeholders no técnicos que deben validar el alcance	Historia de usuario, o descripción breve del caso de uso	Lenguaje natural más accesible; facilita la conversación con perfiles no técnicos.
Actores múltiples con relaciones complejas entre funcionalidades	Caso de uso con diagrama UML (inclusión, extensión, generalización)	El diagrama visualiza relaciones difíciles de mantener como una lista plana de historias.
Proyecto que arranca ágil pero necesita mayor precisión técnica según avanza	Historia de usuario que se desglosa en caso de uso cuando haga falta	Ambas técnicas son complementarias: la historia facilita la comunicación inicial; el caso de uso aporta la precisión que necesita el equipo de desarrollo.

Nota: los apuntes no ofrecen esta tabla de forma explícita; es una síntesis derivada de los apartados sobre la relación entre historias de usuario y casos de uso, y de la nota del programa de la asignatura sobre el uso de casos de uso detallados en entornos regulados. En la práctica, muchos proyectos combinan ambas técnicas (historias de usuario para el backlog general, casos de uso completos solo para las funcionalidades más críticas o complejas), por lo que la decisión rara vez se reduce a aplicar una sola fila de la tabla de forma aislada.

Confusiones habituales

- **Requisito funcional vs. requisito no funcional** — el funcional dice *qué* debe hacer el sistema; el no funcional dice *cómo* debe comportarse (calidad, restricción o interfaz).

- **Restricción real vs. idea de solución** — una restricción legítima responde a una necesidad de fondo verificable; una idea de solución expresada por el usuario (por ejemplo, "una lista desplegable") no es un requisito salvo que esté justificada, y tratarla como tal produce funcionalidad "chapada en oro".
- **Criterios de aceptación vs. Definición de Hecho (DoD)** — los criterios de aceptación son específicos de una historia; la DoD es un acuerdo genérico compartido por todo el proyecto. Cumplir uno no exime de cumplir el otro.
- **Historia de usuario vs. caso de uso** — la historia es un fragmento ligero centrado en el valor para el usuario; el caso de uso es un documento completo con flujo principal, alternativos, precondiciones y postcondiciones.
- **Relación de inclusión (<<include>>) vs. de extensión (<<extend>>)** — la inclusión es obligatoria (el caso de uso siempre necesita al incluido); la extensión es condicional y solo se activa bajo ciertas circunstancias.
- **Actor vs. usuario** — el actor es un rol abstracto definido en el modelo; el usuario es la persona física. Un mismo usuario puede encarnar varios actores según la tarea que realice.
- **Análisis vs. validación de requisitos** — el análisis depura inconsistencias y ambigüedades internas del conjunto de requisitos; la validación comprueba que ese conjunto refleja fielmente las necesidades reales de negocio y usuario.
- **Requisito de usuario (UR) vs. requisito funcional (FR)** — el UR expresa, a alto nivel, qué necesita poder hacer el usuario; el FR lo traduce en un comportamiento específico y verificable del sistema.
- **Detección vs. resolución de conflictos** — la detección identifica que existe un conflicto entre requisitos (mediante análisis comparativo, análisis de dependencias o entrevistas de validación cruzada); la resolución lo negocia con los stakeholders. Confundir una con otra suele resolver de prisa lo que no se ha entendido bien.

Glosario mínimo

Requisito: guía detallada de lo que debe implementarse en un sistema (comportamiento, funciones, atributos y restricciones).

Ingeniería de requisitos: proceso sistemático de elicitación, análisis, especificación, validación y gestión de los requisitos de un sistema.

Analista de requisitos: rol profesional responsable de conducir el proceso de la ingeniería de requisitos y de mediar entre stakeholders.

BO (requisito de negocio): directriz de alto nivel que responde al *por qué* del proyecto.

UR (requisito de usuario): necesidad de un usuario final expresada a alto nivel.

FR (requisito funcional): comportamiento específico y verificable del sistema.

NFR (requisito no funcional): característica o restricción sobre cómo debe comportarse el sistema.

BR (regla de negocio): directriz que gobierna el comportamiento organizativo, con origen en el negocio.

Elicitación: proceso activo de descubrir y construir conocimiento sobre las necesidades de un sistema, no solo de recogerlas.

Stakeholder: cualquier parte interesada con intereses o influencia sobre los requisitos del sistema.

Actor: rol abstracto (persona, sistema o dispositivo) que interactúa con el sistema en un caso de uso.

Caso de uso: descripción, textual y opcionalmente gráfica, de cómo un actor interactúa con el sistema para lograr un objetivo.

Historia de usuario: formato ágil y ligero para capturar una necesidad desde la perspectiva del usuario ("Como... necesito... para...").

Criterios de aceptación: condiciones específicas que determinan que una historia de usuario está completa.

Definición de Hecho (DoD, Definition of Done): acuerdo genérico y compartido por el equipo sobre qué significa que un incremento está "terminado".

Trazabilidad: capacidad de seguir el rastro de un requisito a lo largo de todo su ciclo de vida.

MoSCoW: técnica de priorización que clasifica los requisitos en Must, Should, Could y Won't.

Modelo de Kano: modelo de priorización basado en la satisfacción del usuario, con tres subcategorías principales: *must-be* (básicos), *one-dimensional* (rendimiento) y *delighters* (encantadores).

Escenario: secuencia concreta de pasos (principal o alternativa) dentro de un caso de uso.

Precondición / postcondición: estado del sistema exigido antes de iniciar un caso de uso, o garantizado al finalizarlo.

EBP (Elementary Business Process):

proceso de negocio elemental que aporta valor y deja los datos en un estado consistente; referencia habitual para acotar la granularidad de un caso de uso.

Gold plating: funcionalidad añadida sin que responda a una necesidad real, típica de confundir restricciones con ideas de solución.

CCB (Change Control Board): comité responsable de decidir sobre las solicitudes de cambio de requisitos.

ISO/IEC/IEEE 29148:2018: norma que define las características de calidad que debe cumplir un requisito bien redactado.

Sobre el uso de Inteligencia Artificial en la elaboración de este material

En la elaboración de esta hoja de esenciales y del mapa conceptual que la acompaña se ha utilizado un asistente de Inteligencia Artificial generativa (Claude, Anthropic), en el nivel 3 ("AI Collaboration") de la Escala AIAS (Perkins, Furze, Roe y MacVaugh, 2024), con el siguiente alcance:

- Síntesis, reestructuración y condensación del contenido del Tema 3 —ya redactado y revisado previamente por el profesor— en un formato de repaso: objetivos de aprendizaje, esenciales por bloque, tabla de decisión, confusiones habituales, glosario y mapa conceptual.
- Las siguientes elaboraciones son derivadas de los apuntes y no aparecen así en el texto original: la tabla de decisión sobre historia de usuario frente a caso de uso completo (véase su propia nota al pie); la agrupación de los 15 atributos de calidad en cinco categorías por afinidad (Rendimiento y capacidad, Fiabilidad y continuidad, Seguridad e integridad de los datos, Calidad de uso, Mantenimiento y verificación; los 15 nombres individuales sí proceden literalmente de los apuntes); la agrupación del checklist de 12 comprobaciones en cuatro ejes (Redacción, Contenido, Relación con el resto del sistema, Viabilidad y gestión; los 12 ítems individuales sí proceden literalmente de los apuntes); y la categorización de las nueve técnicas de elicitación del mapa conceptual en tres familias (conversacionales, observacionales, documentales; los nombres y descripciones individuales de las técnicas sí proceden de los apuntes).
- La reenumeración de objetivos y bloques (OA1–OA10, Bloque 1–Bloque 10) es una decisión pedagógica de esta hoja para no confundirlos con la numeración jerárquica de los apuntes; cada bloque indica entre paréntesis su apartado real de correspondencia. No se ha incorporado contenido ajeno a los apuntes fuente ni otro conocimiento externo.

Cualquier duda sobre el alcance de este uso puede consultarse directamente con el profesor.

Mapa conceptual

